UNITÀ 2 · PROGETTARE E UTILIZZARE WEB SERVICE · LEZIONE 6

I web service con SOAP

Una busta XML, un contratto WSDL e tante regole. Il protocollo delle banche, delle assicurazioni e della pubblica amministrazione.

5G TPSI Envelope, Header, Body, Fault, WSDL

PER COMINCIARE

Nel 2026 qualcuno usa ancora SOAP?

SONDAGGIO

- **A** No, è sparito da anni
- **B** Solo qualche sistema vecchio
- **C** Sì, banche e PA ogni giorno
- **D** Non so nemmeno che cosa sia

UN PO' DI STORIA

Simple Object Access Protocol, ma non così semplice

1998

Nasce in Microsoft (Don Box, Dave Winer e altri): RPC in XML su HTTP.

2000

SOAP 1.1 è una Note del W3C. Nel 2001 arriva il WSDL.

2003

SOAP 1.2 è raccomandazione W3C. Il nome non è più un acronimo.

Anni 2000

Le specifiche WS-* (Security, Addressing…) si moltiplicano.

> **Le cose semplici devono essere semplici, quelle complesse devono essere possibili.**
>
> **Alan Kay**, informatico, pioniere della programmazione a oggetti (frase a lui attribuita)

IL MESSAGGIO

Una busta con un'intestazione e un contenuto

soap:Envelope · obbligatorio, la radice

soap:Header · facoltativo

Informazioni per l'infrastruttura: sicurezza, identificativi, transazioni.

soap:Body · obbligatorio

La richiesta o la risposta vera e propria. In caso di errore contiene un **soap:Fault**.

- È un documento **XML**, con il namespace di SOAP sulla radice.
- Di solito viaggia in una richiesta **HTTP POST**, ma può usare anche altri trasporti (code di messaggi, persino email).
- Per SOAP, HTTP è solo un **trasporto**: le informazioni importanti sono tutte dentro la busta.

UN ESEMPIO VERO

Chiedere `somma(20, 22)` in SOAP 1.1

```
POST /calc HTTP/1.1
Content-Type: text/xml; charset=utf-8

<soapenv:Envelope
  xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
  xmlns:c="http://calc.scuola.it/">
  <soapenv:Body>
    <c:somma>
      <a>20</a>
      <b>22</b>
    </c:somma>
  </soapenv:Body>
</soapenv:Envelope>
```

```
HTTP/1.1 200 OK
Content-Type: text/xml; charset=utf-8

<S:Envelope
  xmlns:S="http://schemas.xmlsoap.org/soap/envelope/">
  <S:Body>
    <ns2:sommaResponse
        xmlns:ns2="http://calc.scuola.it/">
      <return>42</return>
    </ns2:sommaResponse>
  </S:Body>
</S:Envelope>
```

Il nome dell'operazione è l'elemento dentro il Body; la risposta si chiama per convenzione **operazione + Response**. I prefissi (`soapenv`, `S`, `ns2`) sono liberi: contano i **namespace**.

METTITI ALLA PROVA

Quali elementi devono esserci in ogni messaggio SOAP?

QUIZ

A Envelope, Header e Body

B Envelope e Body

C Solo il Body

D Body e Fault

L'HEADER

Le informazioni per chi trasporta il messaggio

```
<soapenv:Header>
  <wsse:Security soapenv:mustUnderstand="1"
      xmlns:wsse="http://docs.oasis-open.org/wss/2004/01/oasis-200401-wss-wssecurity-secext-1.0.xsd">
    <wsse:UsernameToken> … </wsse:UsernameToken>
  </wsse:Security>
  <app:IdTransazione xmlns:app="http://registro.example.org/">A-2026-0042</app:IdTransazione>
</soapenv:Header>
```

WS-Security

Credenziali, firme digitali e cifratura di parti del messaggio, anche attraverso più intermediari.

WS-Addressing

A chi va il messaggio, a chi rispondere, che azione è: dentro la busta, non nell'URL.

mustUnderstand

Con il valore 1, chi riceve deve saper gestire quel blocco, altrimenti risponde con un Fault.

IL FAULT

Quando qualcosa va storto

```
HTTP/1.1 500 Internal Server Error
Content-Type: text/xml; charset=utf-8

<S:Envelope xmlns:S="http://schemas.xmlsoap.org/soap/envelope/">
  <S:Body>
    <S:Fault>
      <faultcode>S:Server</faultcode>
      <faultstring>Divisione per zero</faultstring>
    </S:Fault>
  </S:Body>
</S:Envelope>
```

faultcode (1.1)	Significa
`Client`	Richiesta sbagliata: colpa di chi chiama
`Server`	Errore durante l'elaborazione
`VersionMismatch`	Namespace della busta sconosciuto
`MustUnderstand`	Blocco dell'Header non gestito

In SOAP 1.2 gli elementi diventano `Code`, `Reason`, `Detail` e i codici **Sender** e **Receiver**. Su HTTP un Fault viaggia di solito con codice **500**.

DUE VERSIONI

SOAP 1.1 e SOAP 1.2: come riconoscerle

	SOAP 1.1	SOAP 1.2
Namespace della busta	`http://schemas.xmlsoap.org/soap/envelope/`	`http://www.w3.org/2003/05/soap-envelope`
Content-Type	`text/xml`	`application/soap+xml`
Azione	Header HTTP `SOAPAction`	Parametro `action` del Content-Type
Fault	faultcode, faultstring	Code, Reason, Detail
Codici	Client, Server	Sender, Receiver

> Un servizio può offrire entrambe: nel WSDL ci saranno due **binding**. Se mandi una busta 1.2 a un servizio solo 1.1, la risposta è un errore di versione.

QUIZ A TRABOCCHETTO

Con SOAP su HTTP, quale metodo HTTP usa una chiamata a un'operazione?

QUIZ

A GET per leggere, POST per scrivere

B Dipende dall'operazione, come in REST

C PUT, perché si invia un documento

D POST, per tutte le operazioni

WSDL · WEB SERVICES DESCRIPTION LANGUAGE

Il contratto, in XML

```
<definitions name="CalcolatriceService"
    targetNamespace="http://calc.scuola.it/" …>
  <types> schema XSD: somma ha a e b di tipo xs:int </types>
  <message name="somma">
    <part name="parameters" element="tns:somma"/>
  </message>
  <portType name="Calcolatrice">
    <operation name="somma">
      <input message="tns:somma"/>
      <output message="tns:sommaResponse"/>
    </operation>
  </portType>
  <binding name="CalcolatricePortBinding" type="tns:Calcolatrice">
    <soap:binding style="document"
        transport="http://schemas.xmlsoap.org/soap/http"/> …
  </binding>
  <service name="CalcolatriceService">
    <port name="CalcolatricePort" binding="tns:CalcolatricePortBinding">
      <soap:address location="http://localhost:8080/calc"/>
    </port>
  </service>
</definitions>
```

Che cosa (parte astratta): **types**, **message**, **portType** con le operazioni.

Come: il **binding** lega le operazioni a SOAP su HTTP, con stile *document/literal*.

Dove: il **service** con la **port** e l'indirizzo.

> Da un WSDL gli strumenti generano il client in automatico: lo faremo con `wsimport` nella lezione 7.

PUZZLE

Ordina le parti del WSDL, dal più astratto al più concreto

1. **portType** : le operazioni, con messaggio di input e di output
2. **service** : l'insieme delle porte con gli indirizzi
3. **binding** : protocollo e formato per ogni operazione
4. **types** : i tipi di dato, in XML Schema
5. **message** : i messaggi costruiti con quei tipi

AL COMPUTER · 15 minuti

Leggi il WSDL di un servizio pubblico

1. Apri nel browser `https://www.dataaccess.com/webservicesserver/NumberConversion.wso?WSDL`
2. Trova il **targetNamespace** e l'elenco delle **operation** nel portType.
3. Nello schema dentro **types**: come si chiama l'elemento di input di **NumberToWords**? Di che tipo è?
4. In fondo, nel **service**: quante **port** ci sono e qual è l'indirizzo? Quale binding è SOAP 1.2?
5. Rispondi in **wsdl-cognome.txt**.

Controllo: namespace `http://www.dataaccess.com/webservicesserver/`; operazioni **NumberToWords** e **NumberToDollars**; l'input di NumberToWords è **ubiNum**, di tipo `xs:unsignedLong`; due port, una per SOAP 1.1 e una per SOAP 1.2, con lo stesso indirizzo.

AL COMPUTER · 15 minuti

La tua prima chiamata SOAP con curl

```
<!-- numero.xml -->
<soap:Envelope
  xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <NumberToWords
      xmlns="http://www.dataaccess.com/webservicesserver/">
      <ubiNum>2026</ubiNum>
    </NumberToWords>
  </soap:Body>
</soap:Envelope>
```

```
curl -i -H "Content-Type: text/xml; charset=utf-8" --data-binary @numero.xml
  https://www.dataaccess.com/webservicesserver/NumberConversion.wso
```

1. Salva **numero.xml** e lancia curl (tutto su una riga).
2. Prova un altro numero, poi l'operazione **NumberToDollars** con l'elemento **dNum**.
3. Incolla richieste e risposte in **soap-curl-cognome.txt**.

Controllo: **HTTP 200**, Content-Type `text/xml`, e nel Body un elemento **NumberToWordsResult** con il numero in parole inglesi, come *two thousand twenty six*.

AL COMPUTER · 20 minuti

Un server SOAP senza librerie (per capire)

```
import com.sun.net.httpserver.HttpServer;
import java.net.InetSocketAddress;
import java.util.regex.*;

public class MiniSoap {
  static String busta(String corpo) {
    return "<soap:Envelope xmlns:soap="
      + "\"http://schemas.xmlsoap.org/soap/envelope/\">"
      + "<soap:Body>" + corpo + "</soap:Body></soap:Envelope>";
  }
  public static void main(String[] x) throws Exception {
    var srv = HttpServer.create(new InetSocketAddress(8082), 0);
    srv.createContext("/calc", ex -> {
      String q = new String(ex.getRequestBody().readAllBytes());
      Matcher a = Pattern.compile("<a>(-?\\d+)</a>").matcher(q);
      Matcher b = Pattern.compile("<b>(-?\\d+)</b>").matcher(q);
      int code = 500;
      String corpo = "<soap:Fault><faultcode>soap:Client"
        + "</faultcode><faultstring>Richiesta non valida"
        + "</faultstring></soap:Fault>";
      if (q.contains("somma") && a.find() && b.find()) {
        int s = Integer.parseInt(a.group(1))
              + Integer.parseInt(b.group(1));
        code = 200;
        corpo = "<c:sommaResponse xmlns:c=\"http://calc.scuola.it/\">"
          + "<return>" + s + "</return></c:sommaResponse>";
      }
      byte[] out = busta(corpo).getBytes();
      ex.getResponseHeaders().set("Content-Type", "text/xml; charset=utf-8");
      ex.sendResponseHeaders(code, out.length);
      ex.getResponseBody().write(out);
      ex.close();
    });
    srv.start();
  }
}
```

1. Scrivi **MiniSoap.java** e avvialo con `java MiniSoap.java`. **2.** Rispondi: questo server capisce davvero l'XML?

Controllo: resta in attesa sulla porta 8082.

AL COMPUTER · 15 minuti

Richieste buone, richieste cattive, Fault

```
<!-- somma.xml -->
<soapenv:Envelope
    xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
    xmlns:c="http://calc.scuola.it/">
  <soapenv:Body>
    <c:somma><a>20</a><b>22</b></c:somma>
  </soapenv:Body>
</soapenv:Envelope>
```

```
curl -i -H "Content-Type: text/xml; charset=utf-8"
     --data-binary @somma.xml http://localhost:8082/calc
```

1. Salva **somma.xml** e invialo (comando su una riga).
2. Crea **senza-a.xml** togliendo `<a>20</a>` e invialo.
3. Annota codice HTTP e faultstring.

Controllo: con somma.xml **200 OK** e `<return>42</return>`. Senza a: **500** con un **soap:Fault**, faultcode **soap:Client**, "Richiesta non valida".

BILANCIO

SOAP: quando ha senso sceglierlo

Punti di forza

- Contratto formale e tipizzato: il WSDL
- Client generati in automatico, in molti linguaggi
- Sicurezza a livello di messaggio (WS-Security)
- Estensioni standard per affidabilità e transazioni
- Indipendente dal trasporto

Punti deboli

- Messaggi XML lunghi e lenti da elaborare
- Complesso: tante specifiche, strumenti pesanti
- Tutto in POST: niente cache HTTP
- Scomodo da usare dal browser e dalle app
- Poco leggibile per chi lo prova a mano

All'esame: SOAP è una scelta difendibile per integrare sistemi tra aziende o con la PA, dove contano contratto e sicurezza; per un'app web o mobile si preferisce REST con JSON.

OGGI

Contratti per le macchine, ieri e oggi





BIGLIETTO D'USCITA

Completa prima di uscire

L'elemento radice di ogni messaggio SOAP è l'_____.

L'elemento facoltativo per sicurezza e altre informazioni è l'_____.

In caso di errore il Body contiene un _____.

Nel WSDL le operazioni sono elencate nel _____.

L'indirizzo del servizio si trova nell'elemento _____.

⏱ 3:00

RIEPILOGO

Cosa portiamo a casa

- SOAP: messaggi XML in una busta, di solito in POST
- Envelope e Body obbligatori, Header facoltativo
- Fault: faultcode Client o Server, HTTP 500
- 1.1: text/xml e SOAPAction; 1.2: application/soap+xml
- WSDL: types, message, portType, binding, service
- Contratto forte e WS-Security, ma pesante

Hai una domanda su una slide? Usa il pulsante **? Domande** in basso: la vedrà il docente e tutta la classe.